# DevOps & Cloud Engineer – Practical Assessment Tasks

Oct 15, 2026 · by Mario Turic + AI

Two independent take-home tasks, each sized for 20–30 hours of focused work by a junior to middle level  DevOps & Cloud engineer.

Task 1 tests platform building (IaC, Kubernetes, GitOps, security baseline).

Task 2 tests delivery and operations (supply chain, release strategy, observability, incident handling).

## Provided Azure environment

All work runs in one Azure subscription that we provide. `staging` and `prod` must fit into it. Fill in the bracketed values before sending the doc.

| Item | Value |
|---|---|
| Subscription | `<name / subscription ID>` |
| Access | Guest account in tenant `<tenant name>`, MFA required |
| Candidate role | Owner on the subscription, or Contributor + User Access Administrator (needed for managed identities and role assignments) |
| Region | Germany West Central; other regions only with approval |
| Budget | `<€ amount>` for the whole assignment (suggested: €200); budget alerts at 50%, 80% and 100% go to candidate and reviewer |
| vCPU quota | `<n>` vCPUs; size node pools to fit |
| Starting state | Empty subscription; the candidate bootstraps the Terraform state storage account and documents how |
| Environment layout | One shared AKS cluster with a namespace per environment; separate resource groups per environment for databases, Key Vaults and identities (e.g. `rg-shoplite-staging`, `rg-shoplite-prod`); a second cluster only with a written justification |
| Tagging | Every resource tagged `owner=<candidate>`, `task=1` or `task=2`, `env=staging` or `env=prod` |
| Reviewer access | Reviewers hold Reader throughout and Contributor during the Task 2 incident drill |

| Item | Value |
| --- | --- |
| Access period | Until `<date>`; the subscription is wiped afterwards |

## Candidate instructions

- **Time budget:** 20–30 hours per task, over 7–10 calendar days. Log your hours roughly in the README; going over is not a plus and should not be done.
- **Environment:** everything is delivered in the provided Azure environment (see above). Local clusters are fine for experiments, but the submitted solution must run there. Stop the AKS cluster when you are not working on it to protect the budget.
- **Repository:** one Git repository (GitLab or GitHub), shared with the reviewers. Commit history should show how the work evolved; one giant final commit loses points.
- **Tooling:** free choice, but every choice needs a one-line justification in an `docs/adr/` decision record. Open-source or free tiers only.
- **AI assistants:** allowed and favored - you should use Agentic AI setups, AI agents or at least LLMs like Opus or etc. – we will provide you our shared Claude Code subscription for that. You must be able to explain and change every line in the debrief.
- **Secrets:** never commit secrets, kubeconfigs, or state files. Reviewers will scan the history.
- **Unfinished parts:** expected. Write down what you skipped, why, and how you would do it. A clear "not done" beats a hidden gap.

## Submission

- Repository link plus a `README.md` with: architecture overview, how to run it from zero, how to tear it down, known limitations.
- A one-command (or few-command) bootstrap, e.g. `make up` / `make down`.
- A 10-minute recorded or live demo, followed by a 45–60 minute technical debrief by Mario and maybe another Softtribe Engineer

## Task 1 — "Platform from zero": IaC, Kubernetes and GitOps

**Scenario.** A startup runs "ShopLite", a small web shop, on a single VM with `docker compose`. It goes down on every deploy, secrets sit in a `.env` file, and nobody can rebuild the server. You are hired to move it to a reproducible, secure Kubernetes platform with two environments (`staging`, `prod`) that a second engineer could take over tomorrow.

**The application** (build it yourself in any language, keep it under ~300 lines — the app is not graded):

- `api` — HTTP service with `/healthz`, `/readyz`, `/metrics` and one endpoint that reads/writes products in PostgreSQL.
- `worker` — consumes jobs from Redis (or a queue of your choice) and writes results to PostgreSQL.
- `PostgreSQL` and `Redis` — managed service (e.g. Azure Database for PostgreSQL Flexible Server) or in-cluster operator; justify the choice.

**Requirements**

| # | Area | Must have | Est. hours |
|---|---|---|---|
| 1.1 | Infrastructure as Code | Terraform (or OpenTofu) for network, one AKS cluster in the provided subscription, registry, database, identity. Remote state with locking. Modules reused for `staging` and `prod`, differing only by variables. | 6–8 |
| 1.2 | Containerisation | Multi-stage Dockerfiles, non-root user, read-only root filesystem, pinned base images, image < 150 MB. | 1–2 |
| 1.3 | Kubernetes packaging | Helm chart or Kustomize overlays: resource requests/limits, liveness/readiness probes, PodDisruptionBudget, HorizontalPodAutoscaler for `api`. | 3–4 |
| 1.4 | GitOps | Argo CD or Flux reconciles both environments from Git. Promotion staging → prod is a Git change, not a `kubectl` command. App-of-apps or equivalent bootstrap. | 3–4 |
| 1.5 | Secrets | No plaintext secrets in Git. Use Azure Key Vault or HashiCorp Vault with External Secrets Operator, or SOPS / Sealed Secrets. Microsoft Entra Workload ID for pods; | 2–3 |

| # | Area | Must have | Est. hours |
|---|---|---|---|
| | | no client secrets or storage keys. | |
| 1.6 | Ingress & TLS | Ingress controller, cert-manager with Let's Encrypt (staging issuer is fine), using an Azure DNS zone in the subscription or a nip.io hostname on the ingress public IP. | 1–2 |
| 1.7 | Security baseline | Namespaces per environment, default-deny NetworkPolicies with explicit allows, Pod Security Standards `restricted`, RBAC: a read-only role for developers. | 2–3 |
| 1.8 | Docs & operability | README, at least 1 ADR, architecture diagram, a runbook for "rotate the database password" and "restore the database from backup". | 2–3 |

**Acceptance checks the reviewer will run**

- ☐ `terraform plan` on a clean checkout shows no drift after `make up`.
- ☐ Deleting the `api` Deployment by hand: GitOps restores it within 3 minutes.
- ☐ Changing the image tag in the `staging` path deploys to staging only; prod is untouched.
- ☐ `kubectl exec` into `api` cannot reach Redis directly if only `worker` is allowed to.
- ☐ A load test (e.g. `hey` or `k6`, 50 RPS for 2 min) makes the HPA scale `api` out and back in.
- ☐ Rolling update of `api` under load drops no requests (zero 5xx in the load-test output).
- ☐ `git log -p | grep -i password` finds nothing that works.

**Stretch goals (pick any, not required)**

- Policy-as-code with Kyverno or OPA Gatekeeper (e.g. block `:latest`, require limits).
- Cluster autoscaler / node pool sizing, with a short cost estimate per month for prod.
- Terraform checks in CI: `fmt`, `validate`, `tflint`, `checkov` or `trivy config`.
- Backup and restore of the database proven with a real restore, timed.

## Task 2 — "Ship it safely, then keep it alive": supply chain, observability and an incident

**Scenario.** A team ships a payment-notification service. Last quarter they had a compromised dependency, a bad release that took 40 minutes to roll back, and an outage nobody noticed for 2 hours. You build the delivery pipeline and the operations layer, then handle a simulated incident.

**Starting point.** Use a small HTTP service (yours, or any open-source demo app such as `podinfo`) running on AKS in the provided Azure environment (reuse the Task 1 cluster if the candidate does both), with images in Azure Container Registry. The service must expose Prometheus metrics and structured JSON logs. Task 1 is **not** a prerequisite.

### Part A — Secure CI/CD pipeline (≈ 8–10 h)

| # | Must have |
|---|---|
| 2.1 | GitLab CI or GitHub Actions pipeline: lint → unit test → build → scan → sign → publish → deploy, with stages failing fast and caching used deliberately. |
| 2.2 | Generate an SBOM (Syft or equivalent) and scan image + dependencies (Trivy or Grype). Pipeline fails on CRITICAL findings, with a documented, time-limited exception mechanism. |
| 2.3 | Sign images with Cosign (keyless OIDC preferred) and attach the SBOM as an attestation. |
| 2.4 | Cluster admission policy (Kyverno or Sigstore policy-controller) that rejects unsigned images. Show a rejected deploy in the demo. |
| 2.5 | Pipeline runs with short-lived credentials (OIDC federation to Azure / registry), no long-lived secrets in CI variables. |
| 2.6 | Versioning: semantic version or commit-SHA tags, immutable; `:latest` never deployed. |

### Part B — Progressive delivery (≈ 4–6 h)

- Canary or blue/green release with Argo Rollouts or Flagger: 10% → 50% → 100%.
- Automated analysis on error rate and p95 latency from Prometheus; automatic rollback when thresholds are breached.
- Provide a "bad" build (e.g. env flag that returns 20% HTTP 500) and show the rollout aborting on its own in under 5 minutes.

### Part C — Observability and SLOs (≈ 5–7 h)

- Metrics: Prometheus (kube-prometheus-stack or managed Azure Monitor for Prometheus). Logs: Loki or equivalent. Traces: OpenTelemetry to Tempo/Jaeger — at least one traced request path.
- Define two SLOs (availability and latency), e.g. 99.5% of requests succeed and 95% under 300 ms over 30 days.
- Multi-window, multi-burn-rate alerts for both SLOs (Sloth, Pyrra or hand-written rules), routed to a test channel (Slack webhook, e-mail, or Alertmanager UI).
- One Grafana dashboard as code: request rate, error rate, latency percentiles, saturation, error-budget remaining.

### Part D — Incident drill (≈ 3–5 h)

The reviewer applies **one hidden fault** to the candidate's running environment on an agreed day (candidate is told the date, not the fault). Example faults to choose from:

1. A NetworkPolicy change that blocks the service's DNS egress.
2. A memory limit lowered so pods are OOMKilled under load.
3. A ConfigMap change pointing to a wrong downstream URL.
4. An expired or wrong TLS certificate on the ingress.
5. A node cordoned and drained with a PDB that blocks rescheduling.

The candidate must: detect it from their own alerts (not from being told), mitigate it, and within 24 h deliver a blameless postmortem — timeline, detection gap, root cause, customer impact in SLO terms, and 3–5 action items with owners.

### Acceptance checks the reviewer will run

- ☐ Pushing a commit with a known-vulnerable dependency fails the pipeline at the scan stage.
- ☐ `kubectl run` with an unsigned image is denied by admission.
- ☐ CI variables contain no static cloud keys or registry passwords.
- ☐ Deploying the bad build triggers automatic rollback; no manual action needed.
- ☐ Burn-rate alert fires within the expected window for the induced fault.
- ☐ Dashboard and alert rules live in Git and are applied by the pipeline or GitOps.
- ☐ Postmortem time-to-detect and time-to-mitigate match the alert and deploy timestamps.

### Stretch goals (pick any, not required)

- SLSA provenance attestation and its verification at admission.
- Renovate or Dependabot with auto-merge for patch updates gated by the pipeline.
- Synthetic checks (blackbox exporter or k6 cloud-free) from outside the cluster.
- Chaos experiment (Chaos Mesh / LitmusChaos) run as a scheduled job with results on the dashboard.